

AI Usage Log — TDM Insight

Prepared for: CDE2313 Mobile Application Development, Group Project

Assessment: Assessment 2

Institution: Albukhary International University

Covers: All three team members, one section each, on their own branches.

AI Tool Used

Claude Code (Anthropic), model Claude Sonnet 5 — an interactive CLI/IDE coding assistant, used across separate sessions by different team members on their own branches.

Abdulaziz Taju Mohammedyasin

Student ID: AIU24102453

Branch: member1-abdulaziz

Purpose of Use

- Setting up the project from scratch: `CLAUDE.md` project instructions, repository scaffolding matching the Assessment Instructions' required structure, `docs/Implementation_Roadmap.md`, and `docs/Case_Study_Analysis.md`.
- Initializing the actual Android Studio/Gradle project (Phase 1), including diagnosing and fixing real environment problems: an incompatible Gradle/JDK pairing, then an AGP version newer than the installed Android Studio supported.
- Implementing Phase 3 (workflow selection screen, dynamic patient/dose input form, workflow-specific fields) against Member 2's validation and domain models.
- Researching the Vancomycin calculation methodology from the case study's named sources (PhIS TDM Calculator Manual, myTDM Calculator) and standard pharmacokinetics literature, and implementing the calculation engine's formulas (`TdmCalculationEngine.kt`) — including a correction, prompted by the student explicitly asking for the "real" (not simplified) formula, from an initial simplified Pre+Post volume of distribution calculation to the infusion-corrected, back-extrapolated version.
- Wiring the calculation engine into the input form and navigation (Phase 5 integration): running validation, then the engine, then carrying the result to the Results/Explanation screens.
- Drafting handoff files (untracked, never committed by the assistant) for teammates' domain/calculation and results-UI work, at the student's explicit request, so the student could distribute them.
- Ongoing Git workflow support: syncing branches after teammates' merges, diagnosing merge conflicts caused by stale local copies of files teammates had already committed, and switching the team from a branch-per-task model to one persistent branch per member at the student's direction.

Representative Prompts

- "create claude.md file for your instruction with this [...]"
- "i want you to write everything out [...] ..."
- "you can write it but i want more the code to work like the app to work with a real formula"
- "also you have used some simplicity of formulas i want the real formulas"

AI-Generated Suggestions — Adopted

- The three-persistent-branch Git workflow (one per member, plus `main`), and the corresponding rewrite of `CLAUDE.md`'s GITHUB WORKFLOW section.
- Verifying every build/test claim against an actual `./gradlew` run rather than asserting it would work.
- Splitting handoff work into small, individually buildable commits (e.g. domain models, then validation, then the calculation engine, then tests, as separate commits) rather than one large dump.
- Downgrading AGP to the version the team's actual Android Studio installation supports, once diagnosed as the real cause of a broken IDE sync.

AI-Generated Suggestions — Rejected / Corrected

- Claude initially framed the case study's "lecturer-approved authoritative sources" requirement as "the lecturer will provide the equations" — an overstatement not actually present in the source documents. The student challenged this directly, twice, and Claude corrected the claim to what the text actually says.
- Claude's first Pre+Post volume-of-distribution formula used a simplified $V_d = \text{Dose} / (C_{\text{max}} - C_{\text{min}})$ relationship and a K_e calculation that treated the pre-dose and post-dose samples as if they were on one continuous decay curve, ignoring that a dose is administered between them. The student asked for the "real" (not simplified) formula, which prompted a correction to the infusion-corrected, back-extrapolated two-point method — a genuine calculation error, not just a simplification, caught because the student pushed back rather than accepting the first answer.
- Claude proposed writing a "Calculation Method Proposal" document as if it were an assessment requirement; the student pointed out neither source document asks for such a document, and Claude corrected the framing to "a suggestion, not a requirement."
- Claude accidentally reverted the AGP version to conflict with the Android Studio version compatibility, and separately committed an Android project's initial files with a resources folder missing — both were caught by the student asking to double check the commit, and fixed via amend before anything was pushed.

Modifications Made Personally

- Challenged Claude's overstated framing of the case study's "lecturer-approved sources" requirement, twice, until the claim matched what the source document actually said.
- Explicitly asked for the real, infusion-corrected Pre+Post volume-of-distribution formula after reviewing Claude's first, simplified version — the request that surfaced a genuine calculation error, not just a simplification.

- Pushed back on Claude's "Calculation Method Proposal" document being framed as a requirement, when neither source document asks for one.
- Caught an AGP version regression and a commit with a missing resources folder by asking to double check before anything was pushed, and had both fixed via amend first.
- Decided to move the team from a branch-per-task model to one persistent branch per member, and directed the corresponding `CLAUDE.md` rewrite.
- Requested handoff files for teammates' work be drafted as untracked files rather than committed directly, so they could be reviewed and distributed personally rather than landing in git under someone else's name.

Reflection

- I caught a genuine calculation bug (the simplified Vd formula) because I knew enough pharmacokinetics to ask "is this the real formula?" rather than accepting the first answer.
- I corrected Claude's own reading of the assessment documents twice — on what the lecturer actually promised, and on what counted as a real requirement vs. a suggestion.
- I caught two build-breaking mistakes (the AGP regression and the missing resources folder) before they were pushed, by asking to double-check rather than trusting the first commit.

Amir Wuhab Nurhussen

Student ID: AIU24102454

Branch: `member2-amir`

Purpose of Use

The domain models, validation logic, and calculation engine skeleton/formulas under `domain/model/`, `domain/validation/`, and `domain/calculation/` were drafted using Claude Code with Amir present and directing what was needed, then reviewed and committed onto his own branch in small commits matching one logical change each, and opened as the pull requests that brought it into `main`.

Representative Prompts

- "you can write it but i want more the code to work like the app to work with a real formula" (the prompt that took the calculation engine from `NotImplementedError` stubs to a real, sourced implementation)
- "also you have used some simplicity of formulas i want the real formulas" (the first correction to the Pre+Post volume-of-distribution formula)
- "it means just to take the calculations from example now fix that and also you have used some simplicity of formulas i want the real formulas" — paired with a captured worksheet from the actual myTDM Calculator tool, used to check the implementation line-by-line and correct it to match exactly.

AI-Generated Suggestions — Adopted

- The domain model shape (`PatientInfo`, `DoseInfo`, `WorkflowInput` sealed class, `TdmResult`) and the validation approach (reusable field validators, per-workflow validators combining them), committed as-is after review.
- The calculation engine's initial structure: an interface plus `NotImplementedError` stubs naming exactly what clinical information was missing, rather than a guessed formula — committed as the starting point before formulas existed.

AI-Generated Suggestions — Rejected / Corrected

- The calculation engine's Pre+Post volume-of-distribution formula — the file committed as `TdmCalculationEngine.kt` — went through two real corrections, not one: 1. First, a simplified $Vd = \text{Dose} / (C_{\text{max}} - C_{\text{min}})$ was replaced with a more complex infusion-corrected model after being told that was too simplistic. 2. Then that version was replaced again, this time with a simpler $Vd = \text{Dose} / (\text{true } C_{\text{max}} - \text{true } C_{\text{min}})$ using back-extrapolated concentrations, after being checked line-by-line against a captured worksheet from the actual myTDM Calculator tool and found not to match the real source.
- The K_e time calculation was corrected in the same pass because an earlier version omitted the pre-dose sample's own timing offset from the elapsed-time formula.

Modifications Made Personally

- Reviewed the domain model, validation, and calculation engine code handed off from Abdulaziz's session before committing any of it to `member2-amir`.
- Split that work into separate commits by logical layer (domain models, then validation, then the calculation engine skeleton, then tests) rather than committing it as one block.
- Opened the pull requests that brought each piece into `main`.

Reflection

When I reviewed the calculation engine, I realised that the V_d formula had changed more than once. At first, I assumed the formula provided by Claude was correct because it looked reasonable and the code was working structurally. However, after Abdulaziz questioned whether it was the actual formula used by the source, I understood that I could not just rely on the AI's answer, especially because this involved clinical calculations. We compared the calculation with the actual myTDM Calculator worksheet and found that the previous version did not match. This made me realise that reviewing AI-generated code is important, particularly when the code involves formulas where a small mistake can affect the final result.

When I first committed the calculation engine, the main formulas were still `NotImplementedError` stubs. At that stage, I understood that the structure of the calculation engine was being prepared before the actual formulas were confirmed. I was comfortable committing the structure because it clearly showed what calculations still needed to be implemented instead of pretending that incomplete formulas were correct. Later, when the real formulas were added and corrected, I understood better why it was useful to separate the structure from the actual calculation logic.

Since I was involved in directing and reviewing the work with Claude Code rather than personally typing every line of the generated code, I think I need to make sure I understand the code rather than simply saying that I helped create it. For the viva, I would especially review the files in `domain/model/`,

`domain/validation/`, and `domain/calculation/` so that I can explain what each class does, why the validation is separated from the calculation engine, and how the inputs eventually become the final TDM result. The experience also showed me that using AI does not remove the need for understanding and checking the code myself.

Bonson Adem Alo

Student ID: AIU24102383

Branch: `member3-bonson`

Purpose of Use

- Reviewing existing code against `docs/Implementation_Roadmap.md` and `docs/Case_Study_Analysis.md` to check what was and wasn't implemented, phase by phase.
- Debugging Git/branch issues (diverged branches, a stale personal branch missing teammates' merged work, merge conflicts).
- Writing and running Compose UI tests (`androidTest`) and JUnit unit tests, and using an actual Gradle build/emulator to verify them rather than assuming they'd pass.
- Implementing one Phase 6 optional enhancement (export/share a calculated result as plain text).
- Drafting documentation (this file's original Bonson section, and a `README.md` pass) for submission packaging.

Representative Prompts

- "look at the code. know it inside out. then I need you to prepare a commit message for each of the files."
- "check if each and every phase has been implemented from the implementation road map."
- "so step by step, lets start with the things not implemented. start from phase 4 unfinished part."
- "I need you to get started with phase 5. be detail oriented and follow the .md files we curated together"
- "do what is best and recommended and reasonable and ask me questions so that i will confirm for you, step by step" (for choosing a Phase 6 optional enhancement, and again for Phase 7 packaging).

AI-Generated Suggestions — Adopted

- Splitting a batch of uncommitted changes into small, logically separate commits (matching `CLAUDE.md`'s one-feature-per-commit rule) rather than one large commit.
- Merging `origin/main` into `member3-bonson` (rather than rebasing) to safely pull in teammates' work on an already-pushed, shared personal branch.
- The specific UI test coverage added for `ResultsScreen`, `ExplanationScreen`, and a full end-to-end regression test (`EndToEndTest.kt`) driving all three workflows through the real calculation engine.

- The export/share feature's design: a plain (non-Composable) text formatter, kept separate from the UI, for testability — matching the project's existing separation-of-concerns pattern.

AI-Generated Suggestions — Rejected / Corrected

- Claude initially suggested (and the student then committed) a test change renaming a callback parameter reference from `onContinue` to `onCalculated`, based on an incorrect assumption about what had already changed elsewhere in the code, without verifying the actual current function signature. This produced a real compile error, caught later when an actual Gradle build was run (not just assumed). It was corrected — twice, since the first correction also turned out to be based on a stale, unmerged branch state.
- Claude's first phase-completion assessment (Phase 2's calculation engine, Phase 5's integration) was based on reading a stale local Git branch and incorrectly reported both as "not implemented," when teammates had already completed and merged them into `main`. This was caught, corrected, and explained once the actual Git history was checked properly (`git log`, comparing branches) instead of trusting a single local file read.

Modifications Made Personally

- After Claude treated a one-time "commit this" instruction as standing permission to keep committing unprompted, explicitly told it to stop and ask before every commit/push going forward.
- Chose to merge `origin/main` into `member3-bonson` rather than rebase, after Claude laid out the trade-off, since the branch was already pushed and shared with teammates.
- Ran `git pull`, `git commit`, and `git push` yourself throughout, rather than letting the assistant run them.
- Noticed and reported real symptoms (a PR that looked stale, a teammate who couldn't see it) that led to actually finding the repository was private and narrowing down the real cause, rather than accepting "everything looks fine" at face value.
- Reviewed and selected specific commit messages Claude proposed before committing each one, rather than committing everything as a single block.

Reflection

One of the most important things I learned from using Claude Code was that I cannot assume a suggested code change is correct just because it looks reasonable. At one point, Claude assumed that a callback parameter had already been renamed and suggested changing the test to use `onCalculated` instead of `onContinue`. I committed the change, but when I actually ran the Gradle build, it caused a compile error. This showed me the importance of testing the code instead of relying only on Claude's explanation or on the code looking correct.

I also learned that AI can give an incorrect picture of the project if it is looking at outdated information. When I asked Claude to check which phases of the roadmap had been completed, it initially said that the calculation engine and Phase 5 integration were not implemented. However, those changes had already been merged into `main` by my teammates. I had to check the Git history and branch state to understand what had actually happened. This made me realise that when working with Git and multiple branches, I need to verify the repository state myself rather than relying only on what the AI sees in the current working directory.

Another important lesson was learning how to control the way Claude Code worked with Git. At one point, Claude treated my instruction to commit a change as permission to continue committing changes without asking each time. I had to explicitly tell it to stop and ask for confirmation before every commit or push. I also learned that when I said "do what is best," I still needed to be clear about what I was allowing the AI to do. Overall, using Claude Code helped me develop faster, but it also taught me that I need to stay involved in the development process, verify its suggestions, run the project myself, and make the final decisions rather than blindly accepting AI-generated changes.

Declaration

This log records the team's use of **Claude Code** as an AI-assisted development tool during the TDM Insight project.

AI-generated code and suggestions were **reviewed, tested, corrected, rejected, or modified by the team members** where appropriate. The team did not treat AI output as automatically correct, particularly for clinical calculations, project requirements, Git operations, and build-related changes.

Each team member is responsible for reviewing and completing their own **Reflection** section before submission.